

Universidad

Tecnologías de la Información – TI24

Predicción de Incumplimiento de Préstamos mediante Bagging y Random Forest

Tema: Bagging – Random Forest

Dataset: Loan Default Prediction

Fuente: Kaggle

Estudiante: Soledad Aldunate Flores

Docente: Efrain F. Luna

Fecha: Junio 2025

0 Contents

1	Introducción	3
2	Descripción del Dataset	3
2.1	Información general	3
2.2	Variables del dataset	3
2.3	Distribución de la variable objetivo	4
3	Análisis Exploratorio de Datos (EDA)	4
3.1	Estadísticas descriptivas	4
3.2	Distribución de variables numéricas	6
3.3	Boxplots	7
3.4	Variables categóricas vs Default	8
3.5	Matriz de correlación	9
4	Preprocesamiento de Datos	9
4.1	¿Por qué se guardan los datos procesados?	10
4.2	Resumen del preprocesamiento	10
5	Fundamentos Teóricos del Modelo	10
5.1	¿Qué es Bagging?	10
5.2	Base matemática del Bagging	11
5.3	¿Qué es Random Forest?	11
5.4	Criterio de división: Impureza de Gini	11
5.5	Proceso de entrenamiento	12
5.6	Hiperparámetros clave	12
5.7	Ventajas, limitaciones y casos de uso	13
6	Implementación	13
6.1	Librerías utilizadas	13
6.2	Decisiones de diseño	14
7	Modelo Comparativo: Regresión Logística	14
7.1	¿Por qué Regresión Logística?	14
7.2	¿Cómo funciona?	14
8	Resultados	14
8.1	Métricas de Random Forest	15
8.2	Métricas de Regresión Logística	15
8.3	Matrices de Confusión	15

8.4	Curvas ROC	16
8.5	Importancia de variables – Random Forest	17
9	Análisis e Interpretación de Resultados	17
9.1	Comparación entre modelos	17
9.2	Interpretación de la importancia de variables	18
10	Conclusiones	18
11	Referencias Bibliográficas	19

1 Introducción

En el sector financiero, saber de antemano si un cliente va a incumplir el pago de un préstamo es una información sumamente valiosa. Si un banco puede identificar con tiempo a los clientes de mayor riesgo, puede tomar mejores decisiones sobre a quién prestarle dinero y en qué condiciones, reduciendo así las pérdidas.

Este proyecto aborda ese problema usando aprendizaje automático, específicamente un algoritmo llamado **Random Forest**, que pertenece a la familia de métodos de ensamble conocida como **Bagging** (*Bootstrap Aggregating*).

La idea central del Bagging es simple: en lugar de construir un solo modelo que puede equivocarse, se construyen muchos modelos distintos entrenados sobre muestras distintas de los datos, y luego se combina su opinión. Random Forest lleva esto un paso más lejos al también seleccionar aleatoriamente las variables que cada árbol puede usar, haciendo que los árboles sean más independientes entre sí y, en conjunto, más precisos.

El dataset utilizado contiene información de 255 347 préstamos con variables como edad, ingreso, monto del préstamo, puntaje crediticio, tasa de interés, entre otras. La variable objetivo es **Default**: si el cliente incumplió (1) o no (0).

2 Descripción del Dataset

2.1 Información general

El dataset *Loan Default Prediction* está disponible públicamente en Kaggle. Contiene 255 347 registros y 18 columnas (17 variables más la variable objetivo). Cada fila representa un préstamo individual con sus características al momento de ser otorgado.

Table 1: Resumen del dataset

Característica	Valor
Filas totales	255 347
Columnas totales	18
Variables numéricas	9
Variables categóricas	7
Variable objetivo	Default (binaria: 0 / 1)
Valores nulos	Ninguno
Duplicados	Ninguno

2.2 Variables del dataset

Las variables numéricas son: **Age**, **Income**, **LoanAmount**, **CreditScore**, **MonthsEmployed**, **NumCreditLines**, **InterestRate**, **LoanTerm** y **DTIRatio**.

Las variables categóricas son: `Education`, `EmploymentType`, `MaritalStatus`, `HasMortgage`, `HasDependents`, `LoanPurpose` y `HasCoSigner`. La columna `LoanID` es un identificador único que fue eliminada antes del modelado.

2.3 Distribución de la variable objetivo

La variable `Default` está desbalanceada: el 88.4% de los préstamos no tienen incumplimiento (clase 0) y solo el 11.6% sí lo tienen (clase 1). Esto es esperable en datos financieros reales, donde la mayoría de los clientes pagan sus préstamos.

	count	mean	std	min	25%	50%	75%	max
Age	255347.0	43.498306	14.990258	18.0	31.00	43.00	56.00	69.0
Income	255347.0	82499.304597	38963.013729	15000.0	48825.50	82466.00	116219.00	149999.0
LoanAmount	255347.0	127578.865512	70840.706142	5000.0	66156.00	127556.00	188985.00	249999.0
CreditScore	255347.0	574.264346	158.903867	300.0	437.00	574.00	712.00	849.0
MonthsEmployed	255347.0	59.541976	34.643376	0.0	30.00	60.00	90.00	119.0
NumCreditLines	255347.0	2.501036	1.117018	1.0	2.00	2.00	3.00	4.0
InterestRate	255347.0	13.492773	6.636443	2.0	7.77	13.46	19.25	25.0
LoanTerm	255347.0	36.025894	16.969330	12.0	24.00	36.00	48.00	60.0
DTIRatio	255347.0	0.500212	0.230917	0.1	0.30	0.50	0.70	0.9

Figure 1: Distribución de la variable `Default`

3 Análisis Exploratorio de Datos (EDA)

3.1 Estadísticas descriptivas

La tabla siguiente resume las estadísticas principales de las variables numéricas:

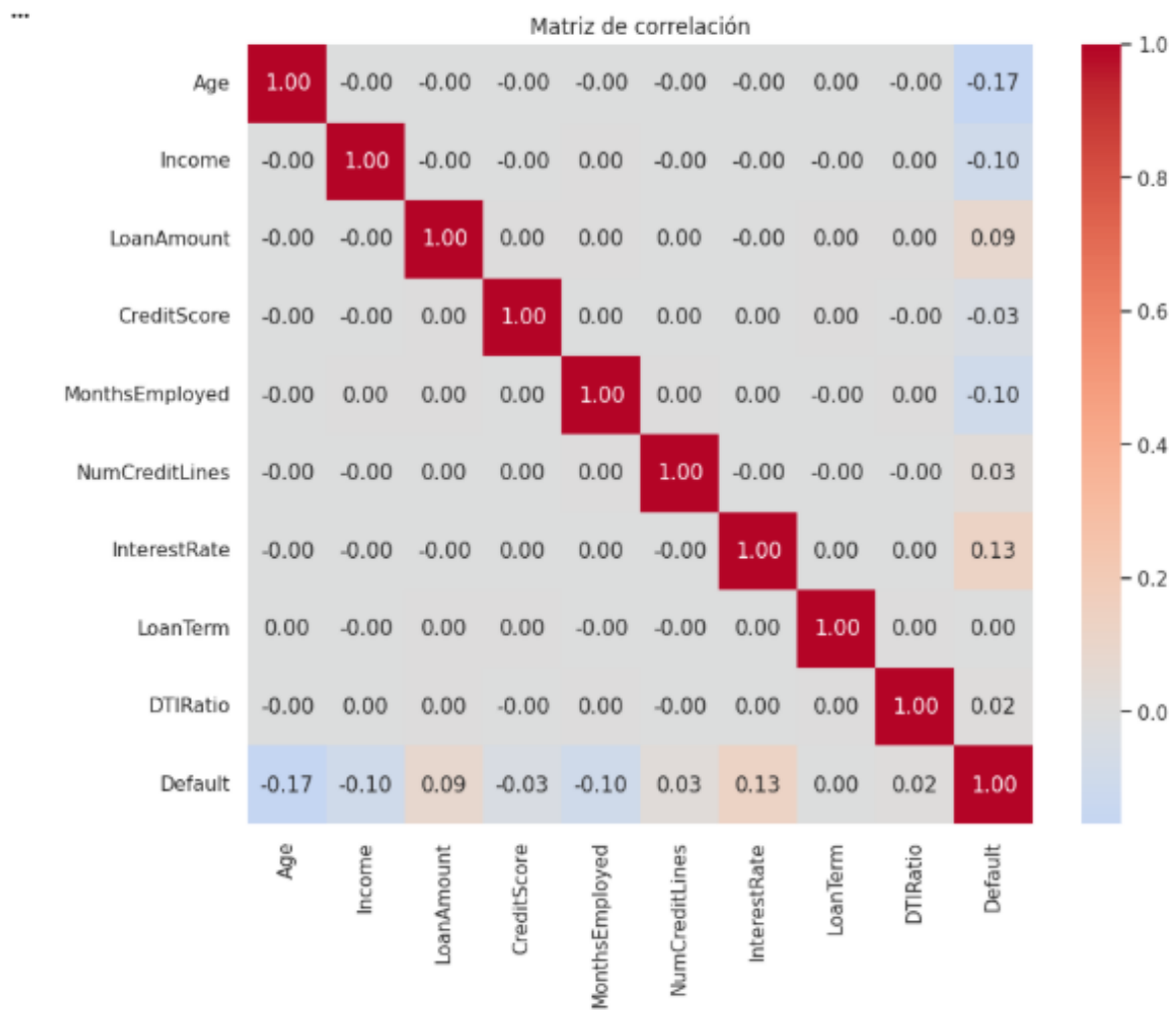


Figure 2: Estadísticas descriptivas de las variables numéricas

Algunos puntos relevantes:

- La edad promedio es de 43.5 años, con un rango de 18 a 69.
- El ingreso promedio es de \$82 499, con alta variabilidad.
- El monto promedio del préstamo es de \$127 578.
- El puntaje crediticio va de 300 a 849, con media en 574.
- El DTI Ratio (relación deuda-ingreso) promedio es 0.50, lo que indica que en promedio la mitad del ingreso se destina a deudas.

3.2 Distribución de variables numéricas

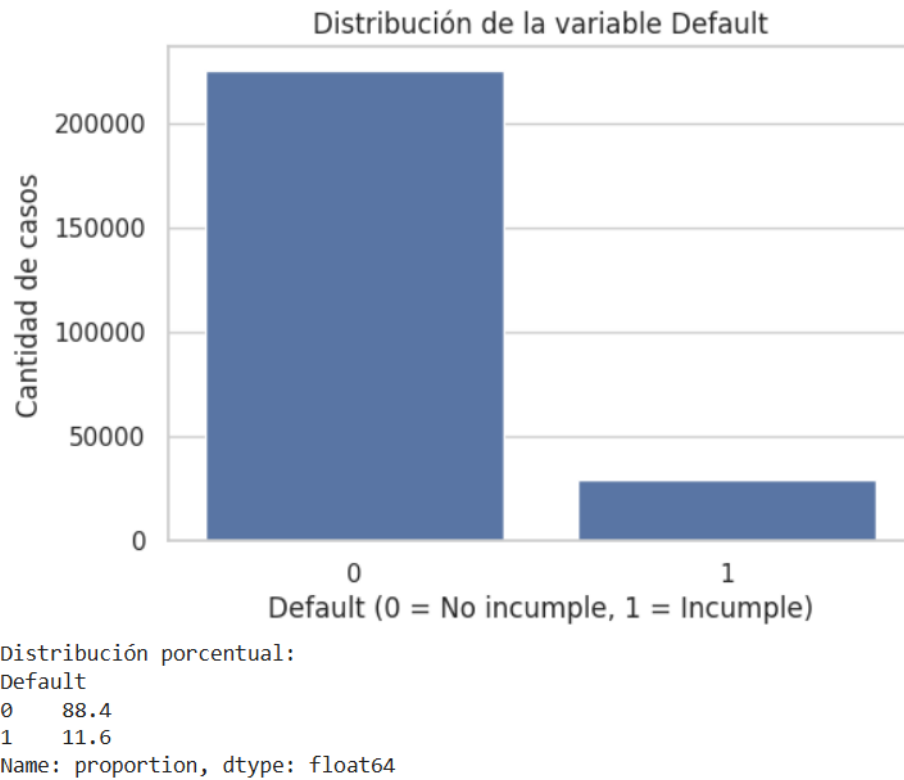


Figure 3: Distribución de las variables numéricas (histogramas)

La mayoría de las variables numéricas muestran distribuciones aproximadamente uniformes, lo que sugiere que el dataset fue generado sintéticamente con distribuciones bien controladas. La variable `NumCreditLines` es discreta (valores 1 a 4).

3.3 Boxplots

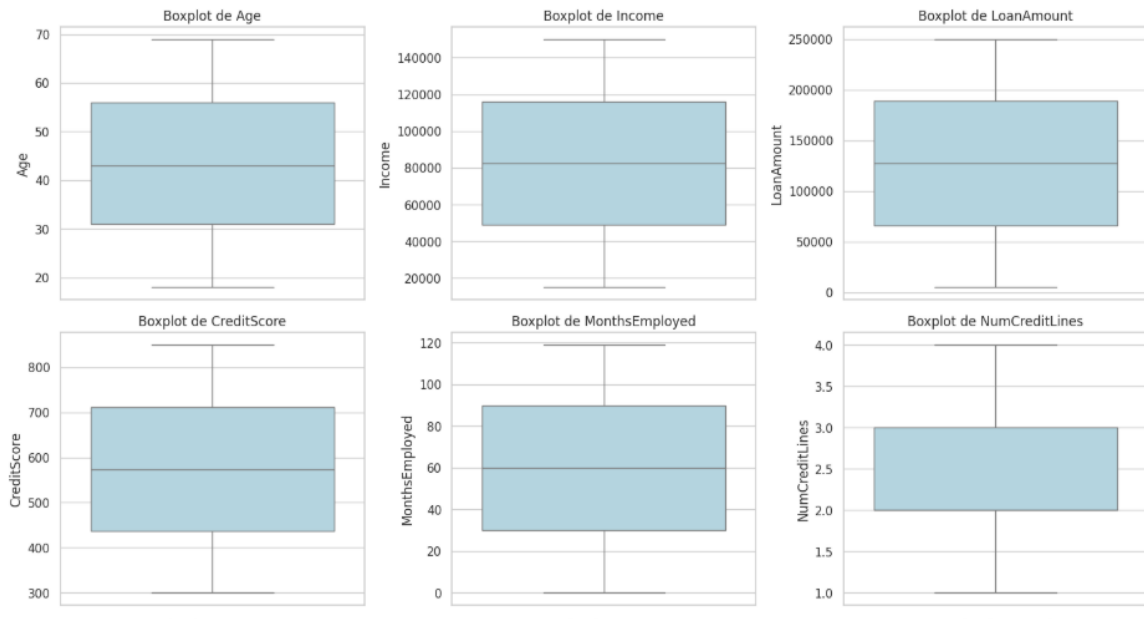


Figure 4: Boxplots de las variables numéricas principales

Los boxplots muestran que ninguna variable presenta valores atípicos extremos. La distribución de cada variable es bastante simétrica, lo que es consistente con la naturaleza sintética del dataset.

3.4 Variables categóricas vs Default

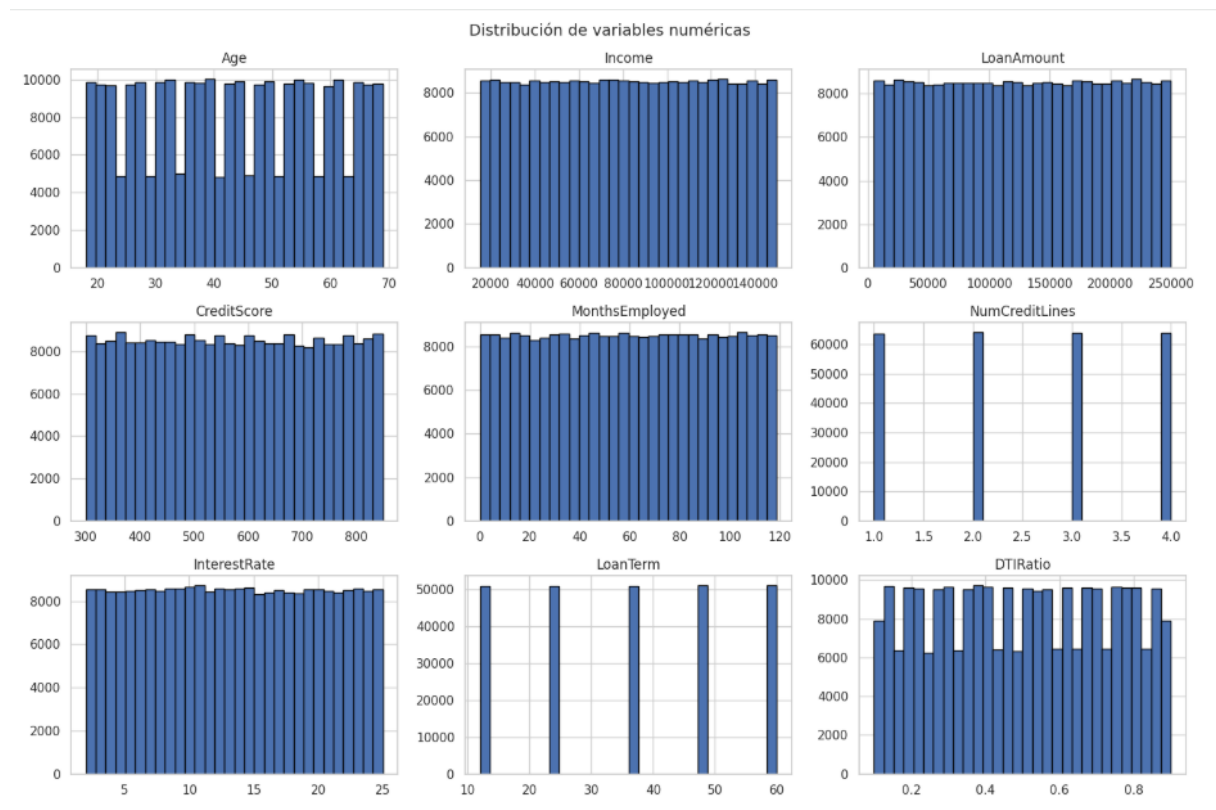


Figure 5: Variables categóricas (Education, EmploymentType, MaritalStatus, HasMortgage) vs Default

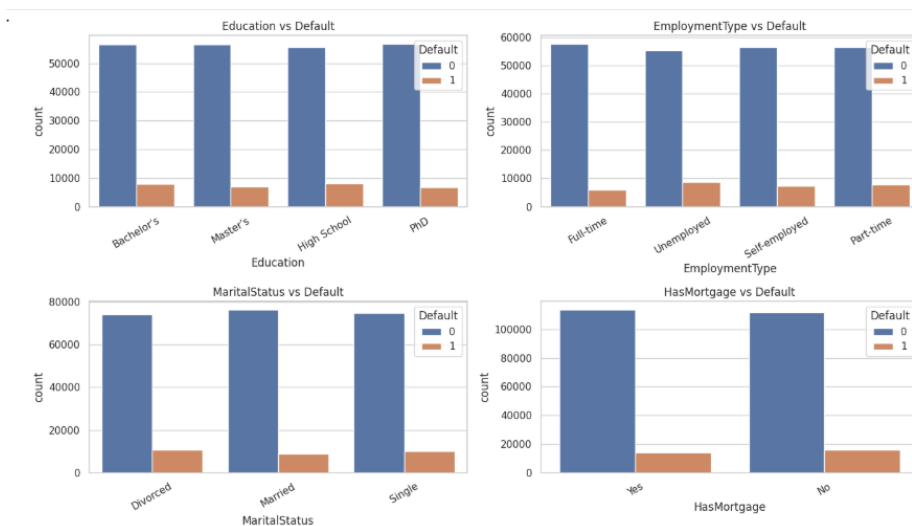


Figure 6: Variables categóricas (HasDependents, LoanPurpose, HasCoSigner) vs Default

En todas las variables categóricas, la proporción de incumplimiento (clase 1) es similar entre categorías, lo que indica que ninguna categoría por sí sola es un predictor fuerte

del default. Esto refuerza la necesidad de usar un modelo que combine muchas variables simultáneamente, como Random Forest.

3.5 Matriz de correlación

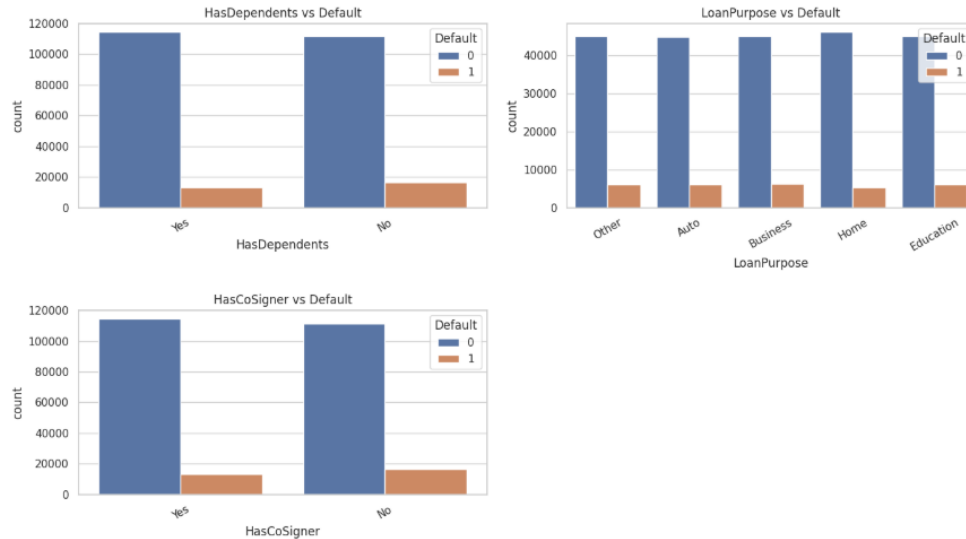


Figure 7: Matriz de correlación entre variables numéricas y Default

Las correlaciones individuales con **Default** son bajas. Las más relevantes son:

- **InterestRate:** correlación positiva de 0.13 (mayor tasa, mayor riesgo).
- **Age:** correlación negativa de -0.17 (mayor edad, menor riesgo).
- **Income:** correlación negativa de -0.10 (mayor ingreso, menor riesgo).

La ausencia de correlaciones fuertes entre las variables predictoras descarta problemas de multicolinealidad severa.

4 Preprocesamiento de Datos

El preprocesamiento se realizó en el notebook `02_preprocessing.ipynb` y consistió en las siguientes etapas:

1. **Eliminación de columna irrelevante:** Se eliminó `LoanID` porque es solo un identificador, no aporta información predictiva.
2. **Verificación de nulos y duplicados:** El dataset no presentó valores nulos ni filas duplicadas, por lo que no fue necesaria ninguna imputación.

3. **Codificación (Label Encoding):** Las 7 variables categóricas fueron convertidas a valores numéricos usando `LabelEncoder` de `scikit-learn`. Por ejemplo, `Education: “Bachelor’s”` $\rightarrow$ 0, `“High School”` $\rightarrow$ 1, `“Master’s”` $\rightarrow$ 2, `“PhD”` $\rightarrow$ 3.
4. **División train/test:** El dataset fue dividido en 80% para entrenamiento (204 277 registros) y 20% para prueba (51 070 registros), usando `random_state=42` para reproducibilidad y `stratify=y` para mantener la proporción de clases.
5. **Escalado (StandardScaler):** Las 9 variables numéricas fueron estandarizadas ($\mu = 0$, $\sigma = 1$) sobre el conjunto de entrenamiento, y la misma transformación fue aplicada al conjunto de prueba (sin recalcular la media ni la desviación, para evitar *data leakage*).

4.1 ¿Por qué se guardan los datos procesados?

Al final del notebook de preprocesamiento se guardan los conjuntos procesados (`X_train.csv`, `X_test.csv`, `y_train.csv`, `y_test.csv`) junto con el escalador (`scaler.pkl`) y los codificadores (`encoders.pkl`) en la carpeta `data/processed/`.

Esto se hace para que los notebooks de modelado (`03_model_main.ipynb` y `04_model_comparison.ipynb`) puedan cargar directamente esos archivos ya listos, sin tener que repetir todo el proceso de preprocesamiento desde cero cada vez. Es decir, actúan como un “punto de partida compartido” para ambos modelos, garantizando que los dos modelos fueron entrenados y evaluados exactamente con los mismos datos.

4.2 Resumen del preprocesamiento

Table 2: Dimensiones finales después del preprocesamiento

Conjunto	Filas	Columnas
X_train	204 277	16
X_test	51 070	16
y_train	204 277	1
y_test	51 070	1

5 Fundamentos Teóricos del Modelo

5.1 ¿Qué es Bagging?

Bagging, cuyo nombre completo es *Bootstrap Aggregating*, es una técnica de aprendizaje automático que busca mejorar la precisión y estabilidad de cualquier modelo al construir varios modelos distintos y luego combinar sus predicciones.

La idea surgió de una pregunta sencilla: si un modelo comete errores porque “aprende de memoria” los datos de entrenamiento (sobreajuste), ¿qué pasa si en lugar de uno entrenamos muchos modelos, cada uno sobre una versión ligeramente diferente de los datos?

El proceso funciona así:

1. Se toman B muestras del conjunto de entrenamiento **con reemplazo** (es decir, un mismo dato puede aparecer más de una vez en la misma muestra, y otros datos pueden no aparecer). Esto se llama *bootstrap*.
2. Con cada muestra se entrena un modelo independiente.
3. Para clasificación, la predicción final es la **votación por mayoría** de todos los modelos.

El resultado es que los errores individuales de cada modelo tienden a cancelarse entre sí, ya que los distintos modelos no cometen los mismos errores en los mismos puntos.

5.2 Base matemática del Bagging

Dado un conjunto de entrenamiento $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$, el proceso de bagging genera B muestras bootstrap $\mathcal{D}_1^*, \dots, \mathcal{D}_B^*$. Sobre cada muestra se entrena un clasificador $h_b(\cdot)$.

La predicción final del modelo de bagging para clasificación es:

$$H(x) = \arg \max_c \sum_{b=1}^B \mathbf{1}[h_b(x) = c] \quad (1)$$

donde $\mathbf{1}[\cdot]$ es la función indicadora que vale 1 si la condición es verdadera y 0 si no lo es. En palabras: la clase ganadora es la que más modelos predican.

5.3 ¿Qué es Random Forest?

Random Forest es una extensión del Bagging que usa árboles de decisión como modelos base y agrega una modificación importante: cuando cada árbol quiere dividir un nodo, **no evalúa todas las variables disponibles**, sino solo una muestra aleatoria de ellas, normalmente $\sqrt{p}$ variables (donde p es el total de variables).

Esto hace que los árboles sean más distintos entre sí (menos correlacionados), lo cual mejora aún más la predicción al combinarlos.

5.4 Criterio de división: Impureza de Gini

Cada árbol en Random Forest decide cómo dividir los datos usando el **índice de impureza de Gini**, que mide qué tan mezcladas están las clases en un nodo. Se define como:

$$G(t) = 1 - \sum_{c=1}^C p_c^2 \quad (2)$$

donde p_c es la proporción de muestras de la clase c en el nodo t . Un nodo perfectamente puro (todas las muestras de una sola clase) tiene $G = 0$. Un nodo completamente mezclado tiene G cercano a 0.5 para dos clases.

La mejor división es la que más reduce la impureza ponderada de los nodos hijos:

$$\Delta G = G(t) - \frac{n_L}{n} G(t_L) - \frac{n_R}{n} G(t_R) \quad (3)$$

donde n_L y n_R son el número de muestras en los nodos izquierdo y derecho respectivamente.

5.5 Proceso de entrenamiento

1. Para cada árbol $b = 1, \dots, B$:
 - (a) Tomar una muestra bootstrap $\mathcal{D}_b^*$ (mismo tamaño que $\mathcal{D}$, con reemplazo).
 - (b) Construir un árbol de decisión profundo sobre $\mathcal{D}_b^*$, donde en cada nodo se evalúan solo $m = \sqrt{p}$ variables aleatorias.
 - (c) Guardar el árbol sin podar.
2. Para predecir una nueva observación x : pasar x por los B árboles y tomar la clase más votada.

5.6 Hiperparámetros clave

Table 3: Hiperparámetros de Random Forest y su impacto

Hiperparámetro	Valor usado	Impacto
<code>n_estimators</code>	100	Número de árboles. Más árboles = más estable, pero más lento.
<code>max_features</code>	<code>sqrt</code>	Variables a evaluar por nodo. Controla la diversidad entre árboles.
<code>max_depth</code>	<code>None</code>	Profundidad máxima. Si es <code>None</code> , los árboles crecen hasta ser puros.
<code>min_samples_split</code>	2	Mínimo de muestras para dividir un nodo.
<code>class_weight</code>	<code>balanced</code>	Ajusta el peso de las clases para compensar el desbalance (88/12).
<code>random_state</code>	42	Semilla para reproducibilidad.

5.7 Ventajas, limitaciones y casos de uso

Ventajas:

- Muy buen rendimiento sin necesidad de mucho ajuste de hiperparámetros.
- Robusto frente a valores atípicos y ruido en los datos.
- Proporciona una medida de importancia de variables muy útil.
- No requiere normalización de los datos (aunque en este proyecto se aplicó por buenas prácticas).
- Reduce el sobreajuste que tendría un solo árbol de decisión.

Limitaciones:

- El modelo final es difícil de interpretar comparado con un solo árbol de decisión.
- Puede ser lento en predicción cuando hay muchos árboles y muchas variables.
- Con datos muy desbalanceados puede seguir teniendo dificultades para detectar la clase minoritaria.

Casos de uso reales:

- Scoring crediticio y detección de fraude en bancos.
- Diagnóstico médico (clasificar si un paciente tiene una enfermedad).
- Detección de spam en correos electrónicos.
- Predicción de abandono de clientes (*churn prediction*).

6 Implementación

6.1 Librerías utilizadas

El proyecto fue desarrollado en Python usando las siguientes librerías principales:

- **pandas** y **numpy**: manipulación y análisis de datos.
- **matplotlib** y **seaborn**: visualizaciones.
- **scikit-learn**: preprocesamiento, modelos y métricas.
- **joblib**: guardado de modelos y transformadores.

6.2 Decisiones de diseño

Se usó `class_weight='balanced'` para que el modelo le preste más atención a los casos de incumplimiento (clase 1), que son la minoría. Sin esto, el modelo tendería a predecir siempre “No Default” porque eso ya le daría un 88% de precisión aparente, pero sería inútil para detectar los casos problemáticos.

También se usó `random_state=42` en el split de datos y en el modelo para que los resultados sean reproducibles.

7 Modelo Comparativo: Regresión Logística

7.1 ¿Por qué Regresión Logística?

Se eligió la Regresión Logística como modelo de comparación porque es el modelo estándar para problemas de clasificación binaria en el ámbito financiero. Es simple, interpretable y sirve como una buena línea base para comparar.

7.2 ¿Cómo funciona?

La Regresión Logística estima la probabilidad de que un préstamo resulte en incumplimiento usando la función sigmoide:

$$P(y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \dots + \beta_p x_p)}} \quad (4)$$

Si la probabilidad supera un umbral (generalmente 0.5), se clasifica como Default (1); de lo contrario, como No Default (0).

La función objetivo que se minimiza durante el entrenamiento es la **entropía cruzada binaria** (*log-loss*):

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)] \quad (5)$$

Se usó también `class_weight='balanced'` para comparar en las mismas condiciones que Random Forest.

8 Resultados

8.1 Métricas de Random Forest

Table 4: Métricas de evaluación – Random Forest

Clase	Precisión	Recall	F1-score	Soporte
No Default (0)	0.93	0.80	0.86	45 139
Default (1)	0.27	0.55	0.36	5 931
Promedio ponderado	0.86	0.77	0.80	51 070
Accuracy			0.797	
AUC-ROC			0.751	

8.2 Métricas de Regresión Logística

Table 5: Métricas de evaluación – Regresión Logística

Clase	Precisión	Recall	F1-score	Soporte
No Default (0)	0.94	0.67	0.78	45 139
Default (1)	0.22	0.70	0.33	5 931
Promedio ponderado	0.87	0.68	0.73	51 070
Accuracy			0.677	
AUC-ROC			0.750	

8.3 Matrices de Confusión

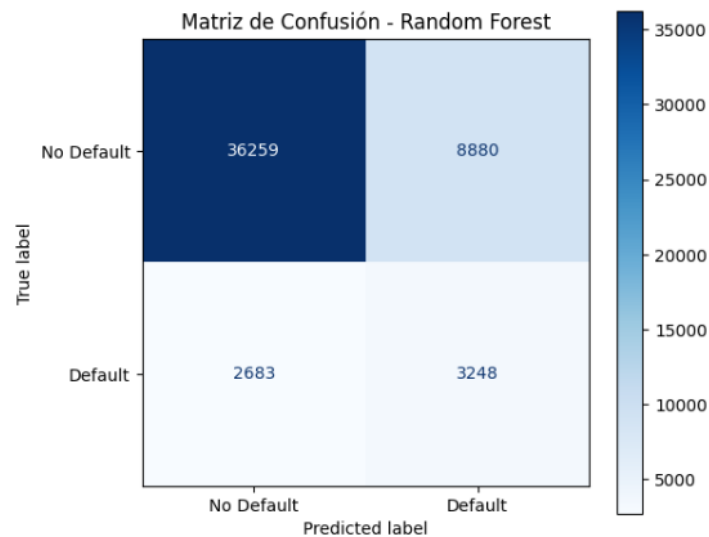


Figure 8: Matriz de confusión – Random Forest

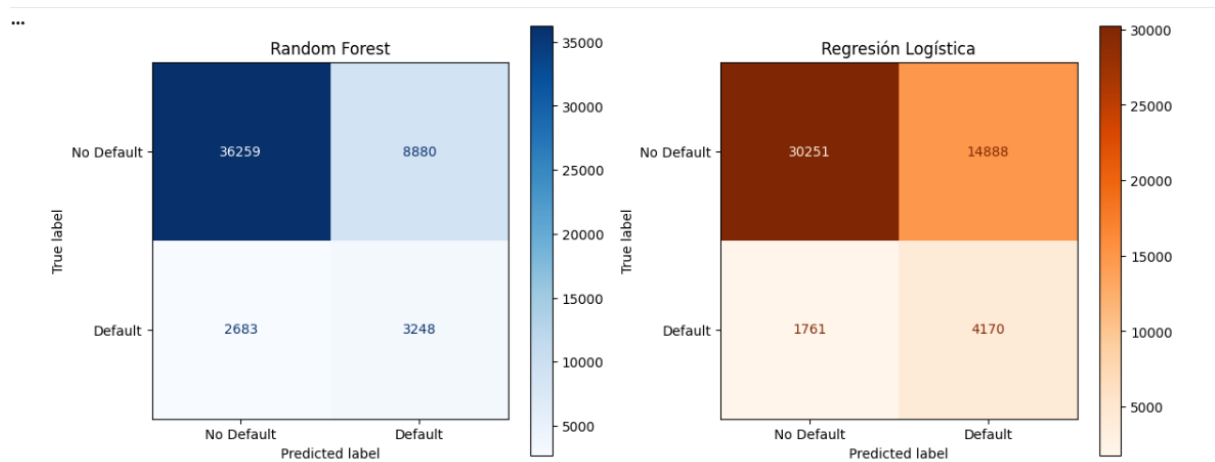


Figure 9: Comparación de matrices de confusión: Random Forest vs Regresión Logística

8.4 Curvas ROC

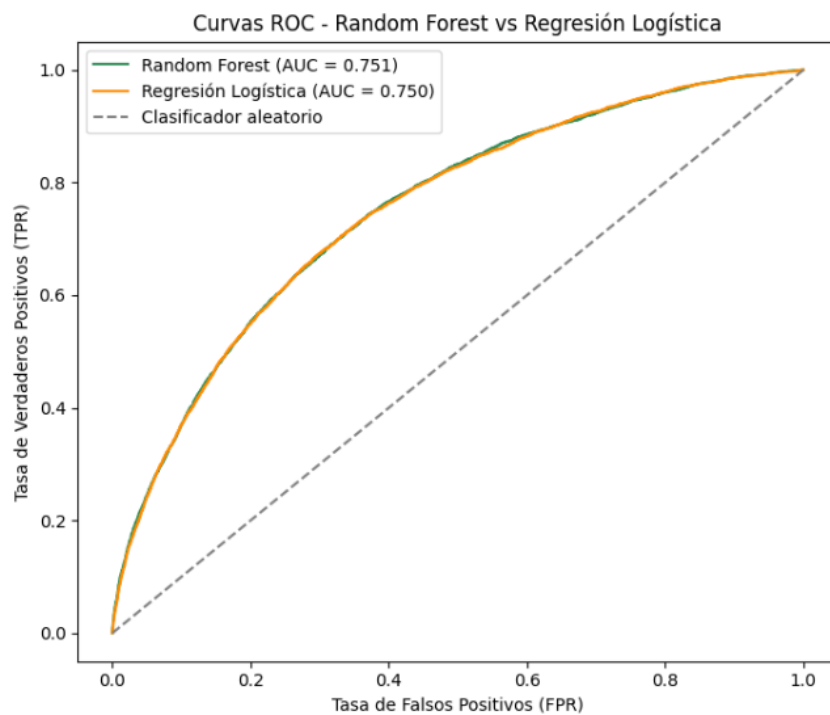


Figure 10: Curvas ROC comparativas (AUC-RF = 0.751, AUC-LR = 0.750)

8.5 Importancia de variables – Random Forest

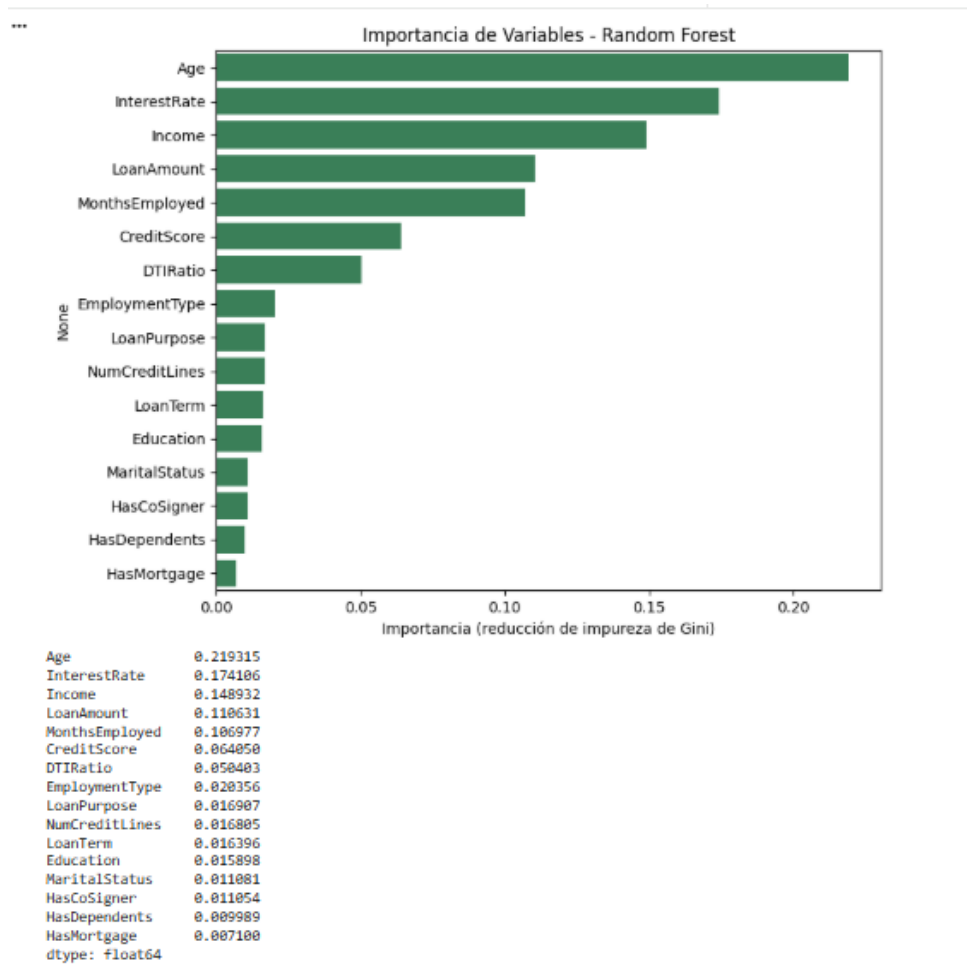


Figure 11: Importancia de variables según reducción de impureza de Gini

Las tres variables más importantes para predecir el incumplimiento son: **Age** (21.9%), **InterestRate** (17.4%) e **Income** (14.9%).

9 Análisis e Interpretación de Resultados

9.1 Comparación entre modelos

Aunque ambos modelos tienen un AUC-ROC casi idéntico (0.751 vs 0.750), lo que significa que ambos tienen una capacidad discriminativa muy similar, Random Forest logra una accuracy general mayor (79.7% vs 67.7%) porque hace menos errores al clasificar los préstamos sin incumplimiento.

Table 6: Comparación resumen entre modelos

Métrica	Random Forest	Regresión Logística
Accuracy	79.7%	67.7%
F1 (No Default)	0.86	0.78
F1 (Default)	0.36	0.33
AUC-ROC	0.751	0.750
Falsos Negativos	2 683	1 761
Verdaderos Positivos	3 248	4 170

Un punto interesante: la Regresión Logística detecta *más* casos reales de default (4 170 vs 3 248 verdaderos positivos), pero a costa de muchos más falsos positivos (14 888 vs 8 880). Es decir, es más “alarmista”: detecta mejor los casos malos pero también rechaza a muchos clientes que en realidad sí pagarían.

9.2 Interpretación de la importancia de variables

La importancia de variables de Random Forest revela que la **edad** del solicitante es el factor más determinante, seguido de la **tasa de interés** y el **ingreso**. Esto tiene sentido:

- Los clientes más jóvenes tienen menos historial y estabilidad financiera.
- Una tasa de interés más alta implica pagos mensuales más altos, lo que aumenta el riesgo de incumplimiento.
- Un ingreso mayor da más capacidad de pago.

Variables como `HasMortgage`, `HasDependents` o `HasCoSigner` tienen muy poca importancia, lo que sugiere que estas características binarias solas no aportan mucha información para distinguir quién incumple.

10 Conclusiones

1. **Random Forest supera a la Regresión Logística** en accuracy general (79.7% vs 67.7%), aunque ambos modelos tienen una capacidad discriminativa similar según el AUC-ROC (≈ 0.75).
2. **El desbalance de clases es el principal desafío.** Con solo 11.6% de casos de default, detectar correctamente los incumplimientos es difícil. El F1-score para la clase Default sigue siendo bajo (0.36), lo que indica que hay margen de mejora.
3. **Las variables más importantes** para predecir el incumplimiento son la edad, la tasa de interés y el ingreso, todas con un significado intuitivo claro.

4. **El Bagging mejora la estabilidad.** Al combinar 100 árboles, el modelo Random Forest es mucho más robusto que un árbol de decisión individual, que tiende a sobreajustarse.

5. **Posibles mejoras futuras:**

- Usar técnicas de balanceo de clases como SMOTE (*Synthetic Minority Over-sampling Technique*).
- Ajustar hiperparámetros con búsqueda exhaustiva (`GridSearchCV` o `RandomizedSearchCV`).
- Probar modelos de gradiente como XGBoost o LightGBM, que suelen tener mejor rendimiento en datos desbalanceados.
- Ajustar el umbral de clasificación para priorizar la detección de defaults.

11 Referencias Bibliográficas

1. Breiman, L. (2001). *Random forests*. *Machine Learning*, 45(1), 5–32. <https://doi.org/10.1023/A:1010933404324>
2. Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction* (2nd ed.). Springer. <https://doi.org/10.1007/978-0-387-84858-7>
3. James, G., Witten, D., Hastie, T., & Tibshirani, R. (2021). *An Introduction to Statistical Learning with Applications in R* (2nd ed.). Springer. <https://www.statlearning.com>
4. Mitchell, T. (1997). *Machine Learning*. McGraw-Hill.
5. Breiman, L. (1996). Bagging predictors. *Machine Learning*, 24(2), 123–140. <https://doi.org/10.1007/BF00058655>